

LAB 8

POST LAB TASKS

1. A system administrator needs to analyze a very large server log file to determine how many requests were handled in a day. Design a multithreaded program where the file is divided into multiple parts, and each thread is responsible for reading its assigned portion and counting the number of lines (each line represents one request). The main thread should collect the results from all threads and compute the total number of requests processed. Care must be taken to ensure that threads do not overlap while reading the file.
2. An e-commerce platform receives thousands of orders in a single day. Each order contains a price value, and the system needs to calculate the total revenue. Write a multithreaded program where the list of orders is divided among multiple threads, and each thread computes the subtotal of its assigned orders. The main thread will combine all subtotals to produce the final revenue. Ensure proper parameter passing to threads.
3. A university wants to analyze student results efficiently. Given a list of student marks, create multiple threads where each thread performs a specific task such as calculating the average marks, finding the highest score, finding the lowest score, and counting the number of students who passed. Each thread should return its computed result to the main thread, which will then display the final statistics.
4. A content moderation system needs to search for a specific keyword within a very large text document. Implement a multithreaded solution where the document is divided into segments, and each thread searches for the keyword in its assigned segment. The total number of occurrences should be calculated by combining the results from all threads. Special attention is required to handle cases where a keyword may span across segment boundaries.
5. A number analysis application is required to process a large list of integers and determine how many of them are prime numbers. Divide the list into equal parts and assign each part to a different thread. Each thread will count

the number of prime numbers in its segment, and the main thread will aggregate these counts to produce the final result.

6. A banking system processes multiple transactions such as deposits and withdrawals on a shared account balance. Create a multithreaded program where each thread represents a transaction. Since synchronization mechanisms are not allowed, observe and discuss how inconsistent results may occur due to concurrent access to the shared balance variable. This task is intended to demonstrate race conditions in a practical scenario.
7. A document processing application needs to count the number of words in multiple text files. Each file should be assigned to a separate thread, which reads the file and counts the words. The main thread will collect the word counts from all threads and display the results for each file as well as the total word count across all files.
8. An online platform stores thousands of product reviews, each with a rating value. Design a multithreaded program where the reviews are divided among threads, and each thread calculates the average rating for its portion. The main thread will then compute the overall average rating of all products. This simulates distributed data processing in real-world systems.
9. A server receives multiple independent tasks, such as calculating the square, cube, or factorial of given numbers. Implement a multithreaded program where each thread processes one or more tasks and stores the result in a shared results array. The main thread should display all processed results after all threads have completed execution.
10. In an election system, votes from different regions need to be counted efficiently. Each region's votes are stored in an array, and each thread is responsible for counting votes from a specific region. After all threads finish execution, the main thread will combine the counts and declare the total votes. This task simulates parallel vote counting in large-scale elections.